IR Project Document

Task 2 – Text Classification Battle

Name: Vino Bala V
Reg No: RA2411026050054
Class: CSE AIML A
Year: 3rd Year

GitHub Repository

https://github.com/vino7075/vino7075-IR_Text_Classification_Vino

Project Overview

This project performs sentiment classification on real customer tweets about six US airlines, from the Twitter US Airline Sentiment dataset. Each tweet is labelled negative, neutral or positive. After removing duplicate and empty tweets, 14,382 tweets were used. The classes are imbalanced: about 63% of the tweets are negative. Each tweet is cleaned by removing @mentions, links and HTML codes, then preprocessed by lowercasing, tokenization, stop word removal and stemming as in Task 1, but with negation words such as 'not' and 'no' kept because they carry sentiment. The text is converted into TF-IDF features (unigrams and bigrams). Decision Tree, KNN, SVM and Naive Bayes are trained on the same stratified 80/20 split and compared using Accuracy, macro Precision, Recall, F1-score and confusion matrices. As a what-if experiment, the standard stop-word list, which removes negation words, was compared with the negation-preserving list.

Code and Execution Screenshots

sentiment_battle.py

```python
"""
Information Retrieval - CLA-2, Task 2
Text Classification Battle: sentiment of airline customer tweets

    Decision Tree  |  K-Nearest Neighbours  |  SVM  |  Naive Bayes

Vino Bala V  (RA2411026050054)  -  III Year CSE AIML A

Dataset : Twitter US Airline Sentiment (Figure Eight / Crowdflower, Feb 2015)
Run     : python sentiment_battle.py
"""

import re
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
                             confusion_matrix, classification_report)

HERE = Path(__file__).parent
RAW_FILE = HERE / "dataset" / "Tweets.csv"
TABLES = HERE / "tables"
FIGS = HERE / "figures"
TABLES.mkdir(exist_ok=True)
FIGS.mkdir(exist_ok=True)

SEED = 11
LABELS = ["negative", "neutral", "positive"]
COLOURS = {"Decision Tree": "#8c564b", "KNN": "#17becf", "SVM": "#9467bd", "Naive Bayes": "#e377c2"}

# Negation words carry the sentiment in phrases like "not happy" or "never again",
# so they are taken OUT of the stop-word list.
NEGATIONS = {"no", "not", "nor", "never", "none", "nothing", "nobody", "nowhere", "cannot", "neither"}
STOP_KEEP_NEGATION = set(ENGLISH_STOP_WORDS) - NEGATIONS
STOP_FULL = set(ENGLISH_STOP_WORDS)

stem = PorterStemmer().stem
console = []


def say(text=""):
    print(text)
    console.append(str(text))
```

Figure 1: Imports, configuration and negation-aware stop-word list

sentiment_battle.py

```python
# ---------------------------------------------------------------------- data
def load_tweets():
    df = pd.read_csv(RAW_FILE, usecols=["airline_sentiment", "airline", "text"])
    df = df.rename(columns={"airline_sentiment": "sentiment"})
    n_raw = len(df)
    df = df.drop_duplicates(subset="text").reset_index(drop=True)
    return df, n_raw - len(df)


def tokenize(tweet, stopwords):
    t = tweet.lower()
    t = re.sub(r"https?://\S+", " ", t)          # links
    t = re.sub(r"@\w+", " ", t)                  # @airline / @user mentions
    t = re.sub(r"&amp;|&lt;|&gt;", " ", t)       # html entities
    t = re.sub(r"n't\b", " not", t)              # didn't -> did not
    words = re.findall(r"[a-z]+", t)
    return " ".join(stem(w) for w in words if w not in stopwords and len(w) > 1)


# ---------------------------------------------------------------------- models
def get_models():
    return {
        "Decision Tree": DecisionTreeClassifier(min_samples_leaf=2, random_state=SEED),
        "KNN": KNeighborsClassifier(n_neighbors=20, metric="cosine"),
        "SVM": LinearSVC(C=0.5, random_state=SEED),
        "Naive Bayes": MultinomialNB(alpha=0.3),
    }


def vectorizer():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)


def fit_and_measure(model, Xa, ya, Xb, yb):
    t0 = time.perf_counter()
    model.fit(Xa, ya)
    t_train = time.perf_counter() - t0
    t0 = time.perf_counter()
    pred = model.predict(Xb)
    t_test = time.perf_counter() - t0
    p, r, f, _ = precision_recall_fscore_support(yb, pred, average="macro", zero_division=0)
    return pred, dict(Accuracy=accuracy_score(yb, pred), Precision=p, Recall=r, F1=f,
                      TrainAcc=accuracy_score(ya, model.predict(Xa)),
                      TrainTime=t_train, TestTime=t_test)
```

Figure 2: Loading data, tweet preprocessing and the four classifiers

sentiment_battle.py

```python
def main():
    df, dups = load_tweets()
    say("SENTIMENT CLASSIFICATION BATTLE - Twitter US Airline Sentiment")
    say("=" * 62)
    say(f"Tweets after removing {dups} duplicates : {len(df)}")
    dist = df["sentiment"].value_counts().reindex(LABELS)
    for lab in LABELS:
        say(f"   {lab:<9}: {dist[lab]:>5}  ({100 * dist[lab] / len(df):.1f}%)")
    say(f"Majority-class baseline accuracy   : {dist.max() / len(df):.4f}")
    dist.to_csv(TABLES / "class_distribution.csv", header=["tweets"])

    df["tokens"] = df["text"].apply(lambda t: tokenize(t, STOP_KEEP_NEGATION))
    empty = int((df["tokens"].str.len() == 0).sum())
    df = df[df["tokens"].str.len() > 0].reset_index(drop=True)
    say(f"Tweets left empty after preprocessing (dropped): {empty}  ->  {len(df)} tweets used")
    say(f"Avg words per tweet : {df['text'].str.split().str.len().mean():.1f} -> "
        f"{df['tokens'].str.split().str.len().mean():.1f} tokens after preprocessing")
    with open(TABLES / "preprocessing_examples.txt", "w") as fh:
        for i in [1, 3, 14]:
            fh.write(f"[{df.sentiment[i]}]\n  tweet : {df.text[i]}\n  tokens: {df.tokens[i]}\n\n")

    idx_tr, idx_te = train_test_split(df.index, test_size=0.20, stratify=df["sentiment"], random_state=SEED)
    y_tr, y_te = df.loc[idx_tr, "sentiment"], df.loc[idx_te, "sentiment"]
    vec = vectorizer()
    X_tr = vec.fit_transform(df.loc[idx_tr, "tokens"])
    X_te = vec.transform(df.loc[idx_te, "tokens"])
    say(f"Train tweets: {len(idx_tr)} | Test tweets: {len(idx_te)} | stratified 80/20")
    say(f"TF-IDF features (unigrams + bigrams): {X_tr.shape[1]}")

    rows, cms, preds = [], {}, {}
    for name, model in get_models().items():
        pred, m = fit_and_measure(model, X_tr, y_tr, X_te, y_te)
        rows.append({"Model": name, **m})
        preds[name] = pred
        cms[name] = confusion_matrix(y_te, pred, labels=LABELS)
        say(f"\n>> {name}")
        say(classification_report(y_te, pred, labels=LABELS, digits=4, zero_division=0).rstrip())

    res = pd.DataFrame(rows)
    res.to_csv(TABLES / "results.csv", index=False)
    say("\nCOMPARISON TABLE (test set, macro averages)")
    say(res.round(4).to_string(index=False))

    recall = pd.DataFrame({n: precision_recall_fscore_support(y_te, p, labels=LABELS, zero_division=0)[1]
                           for n, p in preds.items()}, index=LABELS)
    recall.to_csv(TABLES / "recall_per_class.csv")
    say("\nRecall per class")
    say(recall.round(4).to_string())

    best = res.loc[res["F1"].idxmax(), "Model"]
    say(f"\nBest model (macro F1): {best}")
    cm = cms[best]
    say(pd.DataFrame(cm, index=[f"true {l}" for l in LABELS], columns=[f"pred {l}" for l in LABELS]).to_string())
    for name, c in cms.items():
        pd.DataFrame(c, index=LABELS, columns=LABELS).to_csv(TABLES / f"confusion_{name.replace(' ', '_')}.csv")

    test = df.loc[idx_te, ["sentiment", "text"]].assign(predicted=preds[best])
    test[test.sentiment != test.predicted].to_csv(TABLES / f"misclassified_{best.replace(' ', '_')}.csv",
                                                  index=False)

    feats = np.array(vec.get_feature_names_out())
    svm = LinearSVC(C=0.5, random_state=SEED).fit(X_tr, y_tr)
    with open(TABLES / "svm_top_terms.txt", "w") as fh:
        for i, c in enumerate(svm.classes_):
            fh.write(f"{c:<9}: {', '.join(feats[np.argsort(-svm.coef_[i])[:15]])}\n")

    bar_chart(res)
    confusion_grid(cms)
    recall_chart(recall)
```

Figure 3: Main experiment – split, TF-IDF, training, evaluation and confusion analysis

sentiment_battle.py

```python
    # ---------------- WHAT-IF: what happens if negation words are removed as stop words?
    say("\nWHAT-IF: standard stop-word list (negations removed) vs keeping negation words")
    wi = []
    for label, sw in [("Standard list (negations removed)", STOP_FULL),
                      ("Negations kept (main setup)", STOP_KEEP_NEGATION)]:
        toks = df["text"].apply(lambda t: tokenize(t, sw))
        v = vectorizer()
        A, B = v.fit_transform(toks.loc[idx_tr]), v.transform(toks.loc[idx_te])
        for name, model in get_models().items():
            pred, m = fit_and_measure(model, A, y_tr, B, y_te)
            neg_recall = precision_recall_fscore_support(y_te, pred, labels=LABELS, zero_division=0)[1][0]
            wi.append({"Stop-word list": label, "Model": name, "Accuracy": m["Accuracy"], "F1": m["F1"],
                       "Negative recall": neg_recall})
    wi = pd.DataFrame(wi)
    wi.to_csv(TABLES / "whatif_negation.csv", index=False)
    say(wi.pivot(index="Model", columns="Stop-word list", values="F1").round(4).to_string())
    whatif_chart(wi)

    (TABLES / "console_log.txt").write_text("\n".join(console) + "\n")
    say("\nSaved tables/ and figures/")


if __name__ == "__main__":
    main()
```

Figure 4: What-if experiment code

```
$ python sentiment_battle.py

SENTIMENT CLASSIFICATION BATTLE - Twitter US Airline Sentiment
============================================================
Tweets after removing 213 duplicates : 14427
   negative :  9080  (62.9%)
   neutral  :  3057  (21.2%)
   positive :  2290  (15.9%)
Majority-class baseline accuracy   : 0.6294
Tweets left empty after preprocessing (dropped): 45  ->  14382 tweets used
Avg words per tweet : 17.8 -> 8.2 tokens after preprocessing
Train tweets: 11505 | Test tweets: 2877 | stratified 80/20
TF-IDF features (unigrams + bigrams): 12236

>> Decision Tree
              precision    recall  f1-score   support

    negative     0.7945    0.8413    0.8172      1815
     neutral     0.4801    0.4793    0.4797       605
    positive     0.6154    0.4726    0.5347       457

    accuracy                         0.7066      2877
   macro avg     0.6300    0.5978    0.6105      2877
weighted avg     0.6999    0.7066    0.7014      2877
```

Figure 5: Execution output – dataset details and Decision Tree report

```
$ python sentiment_battle.py

>> SVM
              precision    recall  f1-score   support

    negative     0.8269    0.9157    0.8690      1815
     neutral     0.6423    0.5074    0.5669       605
    positive     0.7635    0.6499    0.7021       457

    accuracy                         0.7876      2877
   macro avg     0.7442    0.6910    0.7127      2877
weighted avg     0.7780    0.7876    0.7790      2877

>> Naive Bayes
              precision    recall  f1-score   support

    negative     0.7527    0.9658    0.8460      1815
     neutral     0.6993    0.3306    0.4489       605
    positive     0.8244    0.4726    0.6008       457

    accuracy                         0.7539      2877
   macro avg     0.7588    0.5897    0.6319      2877
weighted avg     0.7529    0.7539    0.7236      2877

COMPARISON TABLE (test set, macro averages)
        Model  Accuracy  Precision  Recall      F1  TrainAcc  TrainTime  TestTime
Decision Tree    0.7066     0.6300  0.5978 0.6105    0.9027     1.5725    0.0022
          KNN    0.7383     0.6922  0.5991 0.6286    0.7712     0.0187    0.5764
          SVM    0.7876     0.7442  0.6910 0.7127    0.9457     0.0697    0.0007
  Naive Bayes    0.7539     0.7588  0.5897 0.6319    0.8695     0.0186    0.0006

Recall per class
          Decision Tree     KNN     SVM  Naive Bayes
negative         0.8413  0.9212  0.9157       0.9658
neutral          0.4793  0.3488  0.5074       0.3306
positive         0.4726  0.5274  0.6499       0.4726

Best model (macro F1): SVM
              pred negative  pred neutral  pred positive
true negative          1662           116             37
true neutral            243           307             55
true positive           105            55            297
```

Figure 6: Execution output – comparison table, recall per class and best model

```
$ python sentiment_battle.py

WHAT-IF: standard stop-word list (negations removed) vs keeping negation words
Stop-word list  Negations kept (main setup)  Standard list (negations removed)
Model
Decision Tree                        0.6105                             0.5885
KNN                                  0.6286                             0.5965
Naive Bayes                          0.6319                             0.6198
SVM                                  0.7127                             0.6980
```

Figure 7: Execution output – what-if experiment result

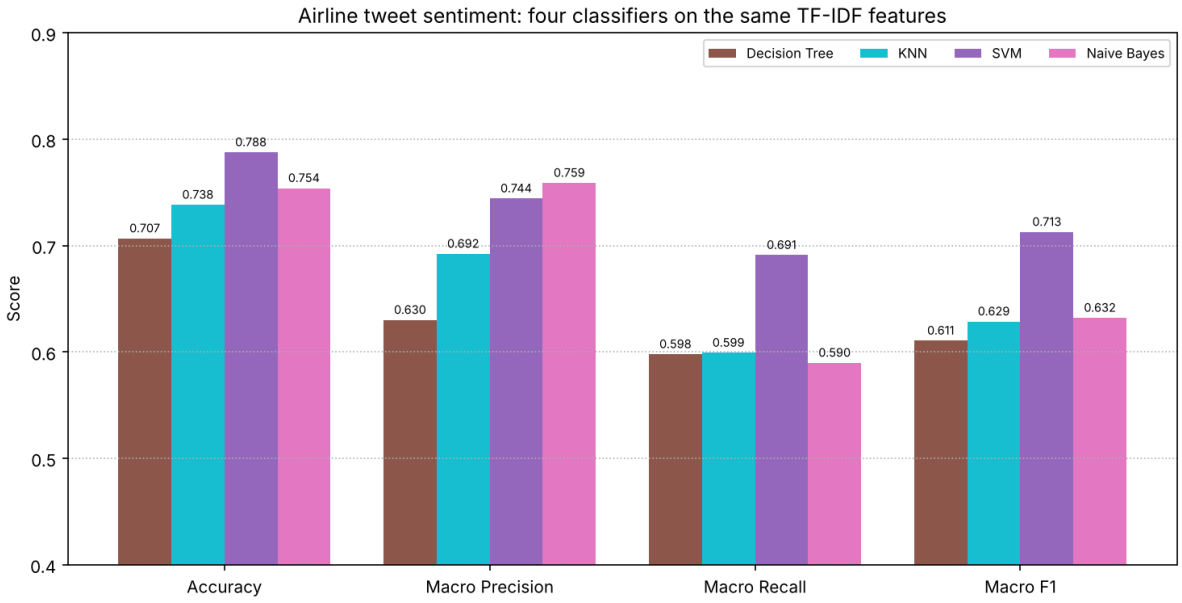


Figure 8: Comparison of the four classifiers

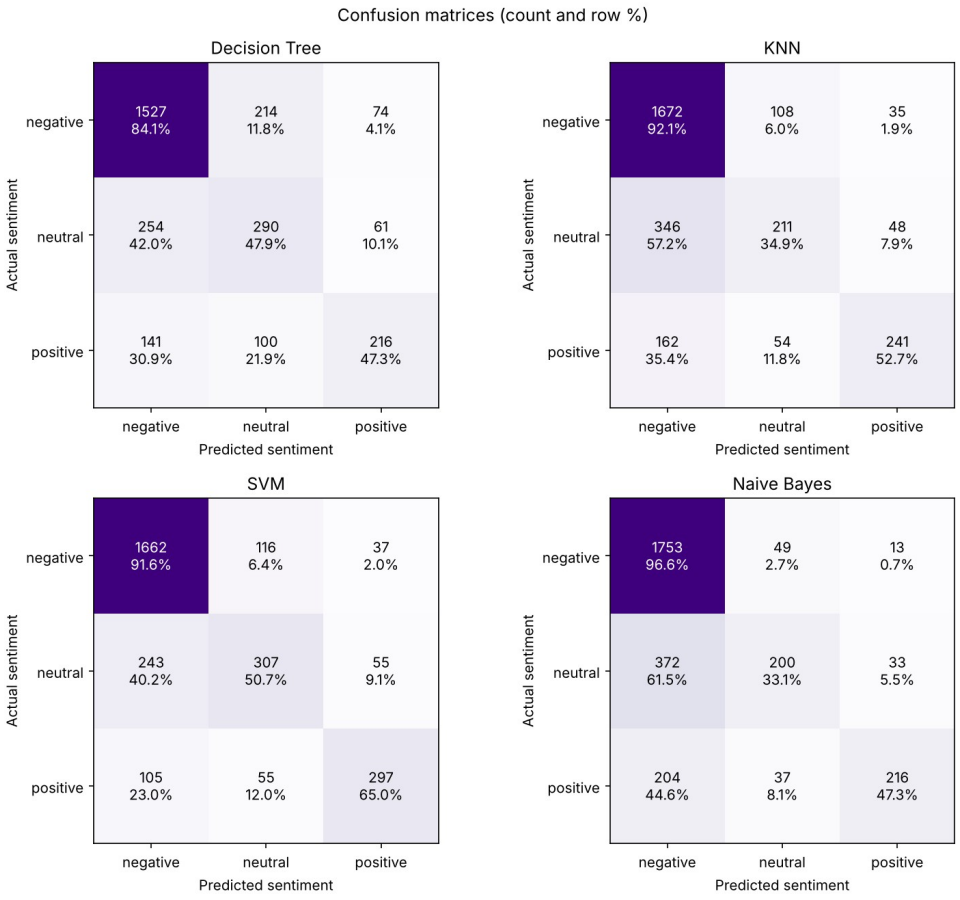


Figure 9: Confusion matrices of the four classifiers

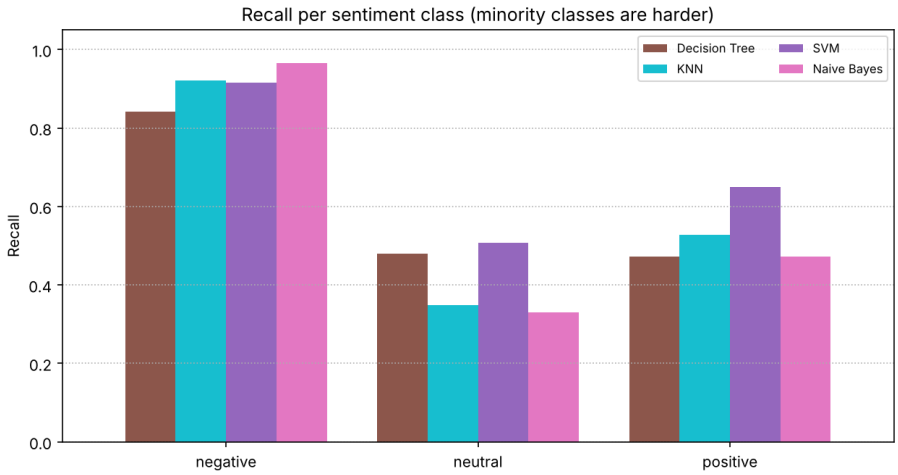


Figure 10: Recall for each sentiment class

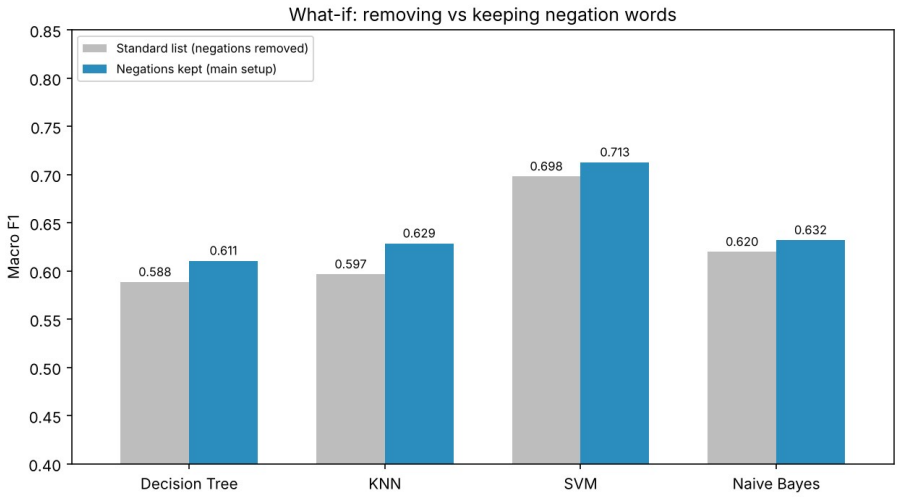


Figure 11: What-if – effect of keeping negation words

Project Result

The program processed 14,382 tweets, built 12,236 TF-IDF features and evaluated all four classifiers on 2,877 test tweets. SVM performed best, with an accuracy of 0.7876 and a macro F1-score of 0.7127. It was followed by Naive Bayes (0.6319), KNN (0.6286) and Decision Tree (0.6105). Because most tweets are negative, every model found the negative class easy (84–97% recall) but struggled with neutral and positive tweets. Naive Bayes predicted 'negative' for 61.5% of neutral tweets. SVM handled the minority classes best, with 65% recall on positive tweets, which is why it has the highest macro F1-score even though its accuracy is not far above the 62.9% majority baseline. In the what-if experiment, removing negation words lowered the macro F1-score of all four models: KNN dropped from 0.6286 to 0.5965 and SVM from 0.7127 to 0.6980. This shows that words like 'not' are important for sentiment. SVM with negation-aware TF-IDF features is therefore recommended for this task.